# Hadoop Prt 1

*Programmazione*

**Apache Hadoop è un framework open-source progettato per gestire e analizzare grandi quantità di dati (Big Data) distribuendoli su più computer collegati in rete.**

**L'idea alla base è semplice: invece di usare un singolo computer molto potente, Hadoop divide i dati e le operazioni tra molti nodi di un cluster, permettendo di elaborare dataset enormi in modo scalabile e relativamente economico.**
**Hadoop nasce nel contesto della crescita esponenziale dei dati digitali (log web, dati social, sensori, transazioni ecc.).**

**Le architetture tradizionali basate su database centralizzati non riuscivano più a gestire volumi così grandi.**

**Hadoop introduce quindi un modello distribuito che permette di archiviare e processare dati su centinaia o migliaia di macchine, mantenendo affidabilità anche se alcuni nodi del sistema falliscono.**

**Un altro aspetto fondamentale di Hadoop è il concetto di fault tolerance: i dati non vengono salvati in una sola posizione ma replicati su più nodi del cluster.**

**Se una macchina si guasta, il sistema continua a funzionare utilizzando le copie presenti su altri nodi.**

## I Componenti Principali di Hadoop

- **HDFS (Hadoop Distributed File System)**

**È il file system distribuito di Hadoop.**
**Permette di salvare grandi file dividendoli in blocchi che vengono distribuiti su diversi nodi del cluster.**

- **MapReduce**

**È il modello di programmazione usato per elaborare i dati in Hadoop.**

**Divide un problema in due fasi principali:**

- **Map → elabora i dati in parallelo su vari nodi**
- **Reduce → aggrega e combina i risultati ottenuti dalle fasi Map**

## I Componenti Principali di Hadoop

- **YARN (Yet Another Resource Negotiator)**

**È il gestore delle risorse del cluster.**
**Coordina l'uso di CPU, memoria e altri elementi del sistema tra le varie applicazioni Hadoop.**

**Funzioni principali:**

- gestione delle risorse
- pianificazione dei job
- coordinamento tra nodi

- **Hadoop Common**

**Contiene le librerie e gli strumenti di base necessari al funzionamento dell'intero ecosistema Hadoop.**

## Quando si usa Hadoop

**Hadoop viene utilizzato soprattutto in scenari di Big Data come:**

- Analisi di log web
- Machine Learning su grandi dataset
- Data mining
- Analisi di dati finanziari
- Elaborazione di dati provenienti da sensori o IoT

## Ecosistema Hadoop

**Intorno a Hadoop si è sviluppato un intero ecosistema di strumenti, tra cui:**

- **Hive → query SQL su Hadoop**
- **HBase → database NoSQL distribuito**
- **Pig → linguaggio di scripting per analisi dati**
- **Spark → motore di elaborazione dati molto più veloce di MapReduce**
- **Sqoop → integrazione con database relazionali**

# Partiamo dal primo esempio: un semplice MapReduce in Java, cioè il linguaggio più classico con cui si introduce Hadoop.

# Esempio base di MapReduce in Hadoop

# L’esempio più famoso e didattico è il WordCount, cioè un programma che legge un testo e conta quante volte compare ogni parola

```
1. import java.io.IOException;
2. import java.util.StringTokenizer;
3.
4. import org.apache.hadoop.conf.Configuration;
5. import org.apache.hadoop.fs.Path;
6. import org.apache.hadoop.io.IntWritable;
7. import org.apache.hadoop.io.Text;
8. import org.apache.hadoop.mapreduce.Job;
9. import org.apache.hadoop.mapreduce.Mapper;
10. import org.apache.hadoop.mapreduce.Reducer;
11. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13.
14. public class WordCount {
15.
16.   // Classe Mapper
17.   // Legge una riga di testo alla volta e produce coppie (parola, 1)
18.   public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
19.
20.     // Valore costante 1 da associare ad ogni parola trovata
21.     private final static IntWritable one = new IntWritable(1);
22.
23.     // Oggetto Text riutilizzato per contenere la parola corrente
24.     private Text word = new Text();
25.
26.     @Override
27.     protected void map(Object key, Text value, Context context) throws IOException, InterruptedException {
28.       // Converte la riga letta in stringa
29.       String line = value.toString();
30.
31.       // Divide la riga in parole usando gli spazi come separatore
32.       StringTokenizer tokenizer = new StringTokenizer(line);
33.
34.       // Per ogni parola trovata emette (parola, 1)
35.       while (tokenizer.hasMoreTokens()) {
36.         word.set(tokenizer.nextToken());
37.         context.write(word, one);
38.       }
39.     }
40.   }
41.
42.   // Classe Reducer
43.   // Riceve una parola e tutti i valori associati, poi li somma
44.   public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
45.
46.     private IntWritable result = new IntWritable();
47.
48.     @Override
49.     protected void reduce(Text key, Iterable<IntWritable> values, Context context)
50.         throws IOException, InterruptedException {
51.
52.       int sum = 0;
53.
54.       // Somma tutti gli 1 associati alla stessa parola
55.       for (IntWritable val : values) {
56.         sum += val.get();
57.       }
58.
59.       // Scrive il risultato finale: (parola, totale)
60.       result.set(sum);
61.       context.write(key, result);
62.     }
63.   }
64.
65.   // Metodo main: configura ed esegue il job Hadoop
66.   public static void main(String[] args) throws Exception {
67.
68.     Configuration conf = new Configuration();
69.     Job job = Job.getInstance(conf, "word count");
70.
71.     // Specifica la classe principale del job
72.     job.setJarByClass(WordCount.class);
73.
74.     // Imposta Mapper e Reducer
75.     job.setMapperClass(TokenizerMapper.class);
76.     job.setReducerClass(IntSumReducer.class);
77.
78.     // Tipo di output prodotto dal Reducer
79.     job.setOutputKeyClass(Text.class);
80.     job.setOutputValueClass(IntWritable.class);
81.
82.     // File di input e cartella di output
83.     FileInputFormat.addInputPath(job, new Path(args[0]));
84.     FileOutputFormat.setOutputPath(job, new Path(args[1]));
85.
86.     // Avvia il job e termina il programma
87.     System.exit(job.waitForCompletion(true) ? 0 : 1);
88.   }
89. }
```

**L'obiettivo è leggere un file di testo e calcolare quante volte compare ogni parola, per farlo Hadoop divide il lavoro in due fasi principali: Map e Reduce, che vengono eseguite in parallelo su diversi nodi del cluster, la prima parte del programma è la classe Mapper.**

**Il suo compito è leggere i dati di input e trasformarli in coppie chiave-valore. In questo caso Hadoop legge il file di testo riga per riga; ogni riga viene convertita in una stringa e suddivisa in parole utilizzando uno tokenizer.**

**Per ogni parola trovata il Mapper produce una coppia composta dalla parola stessa come chiave e dal valore numerico 1.**

**Questo significa che ogni occorrenza della parola viene rappresentata come un piccolo conteggio parziale.**

```java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  // Classe Mapper
  // Legge una riga di testo alla volta e produce coppie (parola, 1)
  public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {

    // Valore costante 1 da associare ad ogni parola trovata
    private final static IntWritable one = new IntWritable(1);

    // Oggetto Text riutilizzato per contenere la parola corrente
    private Text word = new Text();

    @Override
    protected void map(Object key, Text value, Context context) throws IOException, InterruptedException {
      // Converte la riga letta in stringa
      String line = value.toString();

      // Divide la riga in parole usando gli spazi come separatore
      StringTokenizer tokenizer = new StringTokenizer(line);

      // Per ogni parola trovata emette (parola, 1)
      while (tokenizer.hasMoreTokens()) {
        word.set(tokenizer.nextToken());
        context.write(word, one);
      }
    }
  }

  // Classe Reducer
  // Riceve una parola e tutti i valori associati, poi li somma
  public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

      int sum = 0;

      // Somma tutti gli 1 associati alla stessa parola
      for (IntWritable val : values) {
        sum += val.get();
      }

      // Scrive il risultato finale: (parola, totale)
      result.set(sum);
      context.write(key, result);
    }
  }

  // Metodo main: configura ed esegue il job Hadoop
  public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");

    // Specifica la classe principale del job
    job.setJarByClass(WordCount.class);

    // Imposta Mapper e Reducer
    job.setMapperClass(TokenizerMapper.class);
    job.setReducerClass(IntSumReducer.class);

    // Tipo di output prodotto dal Reducer
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // File di input e cartella di output
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    // Avvia il job e termina il programma
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Dopo la fase di Map interviene automaticamente una fase interna di Hadoop chiamata Shuffle and Sort; in questa fase il framework raccoglie tutte le coppie prodotte dai vari Mapper e le raggruppa in base alla chiave.

In pratica tutte le occorrenze della stessa parola vengono raccolte insieme ad esempio: tutte le coppie con chiave “ciao” vengono riunite in un’unica lista di valori associati a quella chiave.

Questo passaggio è fondamentale perché prepara i dati per la fase successiva; la seconda parte del programma è la classe Reducer.
Il Reducer riceve una parola alla volta e l’elenco di tutti i valori associati a quella parola.

Nel nostro caso i valori sono sempre 1, perché il Mapper produceva (parola,1), il Reducer scorre quindi tutti i valori e li somma per ottenere il totale di occorrenze della parola nel dataset.

```
1. import java.io.IOException;
2. import java.util.StringTokenizer;
3.
4. import org.apache.hadoop.conf.Configuration;
5. import org.apache.hadoop.fs.Path;
6. import org.apache.hadoop.io.IntWritable;
7. import org.apache.hadoop.io.Text;
8. import org.apache.hadoop.mapreduce.Job;
9. import org.apache.hadoop.mapreduce.Mapper;
10. import org.apache.hadoop.mapreduce.Reducer;
11. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13.
14. public class WordCount {
15.
16.     // Classe Mapper
17.     // Legge una riga di testo alla volta e produce coppie (parola, 1)
18.     public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
19.
20.         // Valore costante 1 da associare ad ogni parola trovata
21.         private final static IntWritable one = new IntWritable(1);
22.
23.         // Oggetto Text riutilizzato per contenere la parola corrente
24.         private Text word = new Text();
25.
26.         @Override
27.         protected void map(Object key, Text value, Context context) throws IOException, InterruptedException {
28.             // Converte la riga letta in stringa
29.             String line = value.toString();
30.
31.             // Divide la riga in parole usando gli spazi come separatore
32.             StringTokenizer tokenizer = new StringTokenizer(line);
33.
34.             // Per ogni parola trovata emette (parola, 1)
35.             while (tokenizer.hasMoreTokens()) {
36.                 word.set(tokenizer.nextToken());
37.                 context.write(word, one);
38.             }
39.         }
40.     }
41.
42.     // Classe Reducer
43.     // Riceve una parola e tutti i valori associati, poi li somma
44.     public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
45.
46.         private IntWritable result = new IntWritable();
47.
48.         @Override
49.         protected void reduce(Text key, Iterable<IntWritable> values, Context context)
50.                 throws IOException, InterruptedException {
51.
52.             int sum = 0;
53.
54.             // Somma tutti gli 1 associati alla stessa parola
55.             for (IntWritable val : values) {
56.                 sum += val.get();
57.             }
58.
59.             // Scrive il risultato finale: (parola, totale)
60.             result.set(sum);
61.             context.write(key, result);
62.         }
63.     }
64.
65.     // Metodo main: configura ed esegue il job Hadoop
66.     public static void main(String[] args) throws Exception {
67.
68.         Configuration conf = new Configuration();
69.         Job job = Job.getInstance(conf, "word count");
70.
71.         // Specifica la classe principale del job
72.         job.setJarByClass(WordCount.class);
73.
74.         // Imposta Mapper e Reducer
75.         job.setMapperClass(TokenizerMapper.class);
76.         job.setReducerClass(IntSumReducer.class);
77.
78.         // Tipo di output prodotto dal Reducer
79.         job.setOutputKeyClass(Text.class);
80.         job.setOutputValueClass(IntWritable.class);
81.
82.         // File di input e cartella di output
83.         FileInputFormat.addInputPath(job, new Path(args[0]));
84.         FileOutputFormat.setOutputPath(job, new Path(args[1]));
85.
86.         // Avvia il job e termina il programma
87.         System.exit(job.waitForCompletion(true) ? 0 : 1);
88.     }
89. }
```

Il risultato finale viene poi scritto come una nuova coppia chiave-valore composta dalla parola e dal conteggio totale. e lultima parte del codice è il metodo main, che serve a configurare ed eseguire il job Hadoop.

Qui vengono definite le classi da utilizzare come Mapper e Reducer, il tipo di dati prodotti in output e i percorsi dei file di input e output nel file system distribuito.

Quando il job viene avviato, Hadoop distribuisce automaticamente il lavoro tra i nodi del cluster, eseguendo i Mapper in parallelo e successivamente i Reducer sui dati aggregati.

In sintesi, funziona dividendo il problema del conteggio delle parole in due passaggi: prima ogni nodo analizza una parte del testo producendo conteggi parziali, poi Hadoop raggruppa i risultati e il Reducer calcola il totale per ogni parola.

```
1.  import java.io.IOException;
2.  import java.util.StringTokenizer;
3.
4.  import org.apache.hadoop.conf.Configuration;
5.  import org.apache.hadoop.fs.Path;
6.  import org.apache.hadoop.io.IntWritable;
7.  import org.apache.hadoop.io.Text;
8.  import org.apache.hadoop.mapreduce.Job;
9.  import org.apache.hadoop.mapreduce.Mapper;
10. import org.apache.hadoop.mapreduce.Reducer;
11. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13.
14. public class WordCount {
15.
16.     // Classe Mapper
17.     // Legge una riga di testo alla volta e produce coppie (parola, 1)
18.     public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
19.
20.         // Valore costante 1 da associare ad ogni parola trovata
21.         private final static IntWritable one = new IntWritable(1);
22.
23.         // Oggetto Text riutilizzato per contenere la parola corrente
24.         private Text word = new Text();
25.
26.         @Override
27.         protected void map(Object key, Text value, Context context) throws IOException, InterruptedException {
28.             // Converte la riga letta in stringa
29.             String line = value.toString();
30.
31.             // Divide la riga in parole usando gli spazi come separatore
32.             StringTokenizer tokenizer = new StringTokenizer(line);
33.
34.             // Per ogni parola trovata emette (parola, 1)
35.             while (tokenizer.hasMoreTokens()) {
36.                 word.set(tokenizer.nextToken());
37.                 context.write(word, one);
38.             }
39.         }
40.     }
41.
42.     // Classe Reducer
43.     // Riceve una parola e tutti i valori associati, poi li somma
44.     public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
45.
46.         private IntWritable result = new IntWritable();
47.
48.         @Override
49.         protected void reduce(Text key, Iterable<IntWritable> values, Context context)
50.                 throws IOException, InterruptedException {
51.
52.             int sum = 0;
53.
54.             // Somma tutti gli 1 associati alla stessa parola
55.             for (IntWritable val : values) {
56.                 sum += val.get();
57.             }
58.
59.             // Scrive il risultato finale: (parola, totale)
60.             result.set(sum);
61.             context.write(key, result);
62.         }
63.     }
64.
65.     // Metodo main: configura ed esegue il job Hadoop
66.     public static void main(String[] args) throws Exception {
67.
68.         Configuration conf = new Configuration();
69.         Job job = Job.getInstance(conf, "word count");
70.
71.         // Specifica la classe principale del job
72.         job.setJarByClass(WordCount.class);
73.
74.         // Imposta Mapper e Reducer
75.         job.setMapperClass(TokenizerMapper.class);
76.         job.setReducerClass(IntSumReducer.class);
77.
78.         // Tipo di output prodotto dal Reducer
79.         job.setOutputKeyClass(Text.class);
80.         job.setOutputValueClass(IntWritable.class);
81.
82.         // File di input e cartella di output
83.         FileInputFormat.addInputPath(job, new Path(args[0]));
84.         FileOutputFormat.setOutputPath(job, new Path(args[1]));
85.
86.         // Avvia il job e termina il programma
87.         System.exit(job.waitForCompletion(true) ? 0 : 1);
88.     }
89. }
```

**In questo secondo esempio utilizziamo Hadoop per calcolare la lunghezza media delle righe di un file di testo.**

**L'idea è simile al WordCount, ma invece di contare parole andremo a misurare la lunghezza di ogni riga e poi calcolare la media totale.**

**Il Mapper leggerà ogni riga del file e produrrà una coppia chiave-valore dove la chiave sarà sempre la stessa (ad esempio "media") e il valore sarà la lunghezza della riga.**

**In questo modo tutti i valori verranno inviati allo stesso Reducer, che potrà sommarli e calcolare la media finale.**

```java
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageLineLength {

  // Mapper: calcola la lunghezza di ogni riga
  public static class LengthMapper extends Mapper<Object, Text, Text, IntWritable> {

    private Text keyText = new Text("media");

    @Override
    protected void map(Object key, Text value, Context context) throws IOException, InterruptedException {

      String line = value.toString();
      int length = line.length();

      context.write(keyText, new IntWritable(length));
    }
  }

  // Reducer: calcola la media delle lunghezze
  public static class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

      int sum = 0;
      int count = 0;

      for (IntWritable val : values) {
        sum += val.get();
        count++;
      }

      int average = sum / count;

      context.write(new Text("Lunghezza media righe"), new IntWritable(average));
    }
  }

  // Configurazione del job Hadoop
  public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "average line length");

    job.setJarByClass(AverageLineLength.class);

    job.setMapperClass(LengthMapper.class);
    job.setReducerClass(AverageReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Questo programma utilizza ancora il modello MapReduce, ma questa volta il problema da risolvere è il calcolo della lunghezza media delle righe di un file di testo.

La prima parte del programma è il Mapper.

Il suo compito è leggere ogni riga del file e calcolare quanti caratteri contiene. La riga viene convertita in una stringa e viene utilizzato il metodo length() per ottenere il numero di caratteri.

Il Mapper produce quindi una coppia chiave-valore dove la chiave è sempre la parola "media" e il valore è la lunghezza della riga.

L'uso di una chiave unica permette di inviare tutti i valori allo stesso Reducer, che potrà calcolare la media complessiva.

```
1. import java.io.IOException;
2.
3. import org.apache.hadoop.conf.Configuration;
4. import org.apache.hadoop.fs.Path;
5. import org.apache.hadoop.io.IntWritable;
6. import org.apache.hadoop.io.Text;
7. import org.apache.hadoop.mapreduce.Job;
8. import org.apache.hadoop.mapreduce.Mapper;
9. import org.apache.hadoop.mapreduce.Reducer;
10. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
12.
13. public class AverageLineLength {
14.
15.   // Mapper: calcola la lunghezza di ogni riga
16.   public static class LengthMapper extends Mapper<Object, Text, Text, IntWritable> {
17.
18.     private Text keyText = new Text("media");
19.
20.     @Override
21.     protected void map(Object key, Text value, Context context) throws IOException,
    InterruptedException {
22.
23.       String line = value.toString();
24.       int length = line.length();
25.
26.       context.write(keyText, new IntWritable(length));
27.     }
28.   }
29.
30.   // Reducer: calcola la media delle lunghezze
31.   public static class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
32.
33.     @Override
34.     protected void reduce(Text key, Iterable<IntWritable> values, Context context)
35.         throws IOException, InterruptedException {
36.
37.       int sum = 0;
38.       int count = 0;
39.
40.       for (IntWritable val : values) {
41.         sum += val.get();
42.         count++;
43.       }
44.
45.       int average = sum / count;
46.
47.       context.write(new Text("Lunghezza media righe"), new IntWritable(average));
48.     }
49.   }
50.
51.   // Configurazione del job Hadoop
52.   public static void main(String[] args) throws Exception {
53.
54.     Configuration conf = new Configuration();
55.     Job job = Job.getInstance(conf, "average line length");
56.
57.     job.setJarByClass(AverageLineLength.class);
58.
59.     job.setMapperClass(LengthMapper.class);
60.     job.setReducerClass(AverageReducer.class);
61.
62.     job.setOutputKeyClass(Text.class);
63.     job.setOutputValueClass(IntWritable.class);
64.
65.     FileInputFormat.addInputPath(job, new Path(args[0]));
66.     FileOutputFormat.setOutputPath(job, new Path(args[1]));
67.
68.     System.exit(job.waitForCompletion(true) ? 0 : 1);
69.   }
70. }
```

Dopo la fase di Map, Hadoop esegue automaticamente la fase di Shuffle and Sort, nella quale tutte le coppie con la stessa chiave vengono raggruppate.

Poiché tutte le coppie hanno la stessa chiave "media", il sistema raccoglierà tutte le lunghezze delle righe in un’unica lista.

La seconda parte del programma è il Reducer.

Qui il Reducer riceve la chiave "media" e una lista di valori che rappresentano le lunghezze di tutte le righe analizzate.

Il Reducer scorre tutti questi valori e calcola due informazioni: la somma totale delle lunghezze e il numero totale di righe, una volta ottenuti questi due valori, calcola la media dividendo la somma per il numero di righe, il risultato finale viene scritto come output del job Hadoop.

```
1.  import java.io.IOException;
2.
3.  import org.apache.hadoop.conf.Configuration;
4.  import org.apache.hadoop.fs.Path;
5.  import org.apache.hadoop.io.IntWritable;
6.  import org.apache.hadoop.io.Text;
7.  import org.apache.hadoop.mapreduce.Job;
8.  import org.apache.hadoop.mapreduce.Mapper;
9.  import org.apache.hadoop.mapreduce.Reducer;
10. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
12.
13. public class AverageLineLength {
14.
15.   // Mapper: calcola la lunghezza di ogni riga
16.   public static class LengthMapper extends Mapper<Object, Text, Text, IntWritable> {
17.
18.     private Text keyText = new Text("media");
19.
20.     @Override
21.     protected void map(Object key, Text value, Context context) throws IOException,
    InterruptedException {
22.
23.       String line = value.toString();
24.       int length = line.length();
25.
26.       context.write(keyText, new IntWritable(length));
27.     }
28.   }
29.
30.   // Reducer: calcola la media delle lunghezze
31.   public static class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
32.
33.     @Override
34.     protected void reduce(Text key, Iterable<IntWritable> values, Context context)
35.         throws IOException, InterruptedException {
36.
37.       int sum = 0;
38.       int count = 0;
39.
40.       for (IntWritable val : values) {
41.         sum += val.get();
42.         count++;
43.       }
44.
45.       int average = sum / count;
46.
47.       context.write(new Text("Lunghezza media righe"), new IntWritable(average));
48.     }
49.   }
50.
51.   // Configurazione del job Hadoop
52.   public static void main(String[] args) throws Exception {
53.
54.     Configuration conf = new Configuration();
55.     Job job = Job.getInstance(conf, "average line length");
56.
57.     job.setJarByClass(AverageLineLength.class);
58.
59.     job.setMapperClass(LengthMapper.class);
60.     job.setReducerClass(AverageReducer.class);
61.
62.     job.setOutputKeyClass(Text.class);
63.     job.setOutputValueClass(IntWritable.class);
64.
65.     FileInputFormat.addInputPath(job, new Path(args[0]));
66.     FileOutputFormat.setOutputPath(job, new Path(args[1]));
67.
68.     System.exit(job.waitForCompletion(true) ? 0 : 1);
69.   }
70. }
```

**Il metodo main ha il compito di configurare il job Hadoop.**

**In questa parte vengono definiti il Mapper, il Reducer, i tipi di dati usati come output e i percorsi dei file di input e output.**

**Una volta avviato il job, Hadoop distribuisce automaticamente l'elaborazione tra i nodi del cluster, permettendo di analizzare grandi quantità di dati in parallelo.**

**Questo esempio mostra come MapReduce non sia utile solo per contare elementi come nel WordCount, ma anche per eseguire operazioni statistiche distribuite, come somme, medie o aggregazioni su dataset molto grandi.**

```
1.  import java.io.IOException;
2.
3.  import org.apache.hadoop.conf.Configuration;
4.  import org.apache.hadoop.fs.Path;
5.  import org.apache.hadoop.io.IntWritable;
6.  import org.apache.hadoop.io.Text;
7.  import org.apache.hadoop.mapreduce.Job;
8.  import org.apache.hadoop.mapreduce.Mapper;
9.  import org.apache.hadoop.mapreduce.Reducer;
10. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
12.
13. public class AverageLineLength {
14.
15.   // Mapper: calcola la lunghezza di ogni riga
16.   public static class LengthMapper extends Mapper<Object, Text, Text, IntWritable> {
17.
18.     private Text keyText = new Text("media");
19.
20.     @Override
21.     protected void map(Object key, Text value, Context context) throws IOException,
    InterruptedException {
22.
23.       String line = value.toString();
24.       int length = line.length();
25.
26.       context.write(keyText, new IntWritable(length));
27.     }
28.   }
29.
30.   // Reducer: calcola la media delle lunghezze
31.   public static class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
32.
33.     @Override
34.     protected void reduce(Text key, Iterable<IntWritable> values, Context context)
35.         throws IOException, InterruptedException {
36.
37.       int sum = 0;
38.       int count = 0;
39.
40.       for (IntWritable val : values) {
41.         sum += val.get();
42.         count++;
43.       }
44.
45.       int average = sum / count;
46.
47.       context.write(new Text("Lunghezza media righe"), new IntWritable(average));
48.     }
49.   }
50.
51.   // Configurazione del job Hadoop
52.   public static void main(String[] args) throws Exception {
53.
54.     Configuration conf = new Configuration();
55.     Job job = Job.getInstance(conf, "average line length");
56.
57.     job.setJarByClass(AverageLineLength.class);
58.
59.     job.setMapperClass(LengthMapper.class);
60.     job.setReducerClass(AverageReducer.class);
61.
62.     job.setOutputKeyClass(Text.class);
63.     job.setOutputValueClass(IntWritable.class);
64.
65.     FileInputFormat.addInputPath(job, new Path(args[0]));
66.     FileOutputFormat.setOutputPath(job, new Path(args[1]));
67.
68.     System.exit(job.waitForCompletion(true) ? 0 : 1);
69.   }
70. }
```

**Buon Davante a tutti**